

BYTEBURGUER CLOUD PLATFORM: PROPOSTA DE ARQUITETURA, SEGURANÇA E GESTÃO

Aluno: [Nome do Estudante]

Disciplina: Segurança da Informação, Gestão de Projetos, Engenharia de Software, DevOps, Arquitetura de Computadores

Professor: [Nome do Professor]

Instituição: [Nome da Instituição]

Curso: [Nome do Curso]

Data: Junho de 2026

FOLHA DE ROSTO

Trabalho acadêmico apresentado como requisito parcial para a avaliação da disciplina de Segurança da Informação e Engenharia de Software, no curso de [Nome do Curso], na [Nome da Instituição].

Orientador: [Nome do Orientador]

Aluno: [Nome do Estudante]

Cidade, 2026.

SUMÁRIO

1. Introdução
2. Desenvolvimento
 - 2.1. Segurança e Auditoria
 - 2.2. Gestão de Projetos
 - 2.3. Projeto de Software
 - 2.4. Arquitetura e Organização de Computadores
3. Conclusão
4. Referências Bibliográficas

1 Introdução

A ByteBurger atua em um cenário operacional que exige sinergia entre automação, segurança da informação e governança corporativa. A companhia mantém restaurantes automatizados em diversas cidades, operando robôs-cozinheiros,

sensores IoT, totens de autoatendimento, aplicativos móveis, sistemas administrativos e dashboards gerenciais. A infraestrutura apresentada é híbrida, com componentes distribuídos entre Edge e Cloud. Essa complexidade aumenta a superfície de risco e demanda uma proposta arquitetural que simultaneamente atenda aos requisitos de confiabilidade, disponibilidade, segurança e compliance. A escolha de abordar a ByteBurger como um caso de estudo também revela a necessidade de colocar em prática conceitos teóricos de gestão de projetos, engenharia de software, DevOps e arquitetura de computadores.

O objetivo deste trabalho é construir uma solução integrada para a ByteBurger Cloud Platform, capaz de resolver problemas identificados no ambiente atual: robôs ativados fora do horário, sensores com dados inconsistentes, pedidos desaparecendo, falhas de autenticação, acessos indevidos, logs alterados ou inexistentes, falta de rastreabilidade, ausência de auditoria, dados sensíveis sem proteção adequada e divergências entre franquias. Essas falhas não são isoladas; elas refletem fragilidades em políticas de identidade, arquitetura de dispositivos, proteção de dados e governança. Assim, o trabalho se estrutura em quatro grandes desafios técnicos e gerenciais, que se complementam para criar uma solução robusta e alinhada às disciplinas do semestre.

No primeiro desafio, dedicado a Segurança e Auditoria, o foco é implementar um modelo de identidade centralizado, um contexto seguro para IoT e robótica, proteção de dados e monitoramento contínuo. A literatura técnica indica que a integração de IAM, RBAC, MFA, SSO e Vault é essencial para reduzir riscos de acessos indevidos (Imoniana, 2016). Enquanto isso, a arquitetura de PKI, certificados digitais e MQTT seguro oferece a base para comunicação confiável com dispositivos IoT, como recomendado por Pressman e Maxim (2021) e por Sommerville (2018). A proposta destaca também o uso de TLS 1.3, segmentação de rede, Zero Trust, controle de firmware e atualizações OTA para garantir que robôs e sensores operem dentro de padrões de segurança.

O segundo desafio enfatiza Gestão de Projetos e contempla escopo, EAP com quatro níveis, cronograma, matriz de riscos, plano de comunicação e escolha metodológica. A escolha por um modelo híbrido de Scrum, Kanban, XP e DevOps não é apenas uma questão de moda; é uma decisão fundamentada para permitir adaptação contínua em um ambiente com múltiplos stakeholders e requisitos emergentes, conforme discutido por Kogon, Blakemore e Wood (2019), Maximiano e Veroneze (2022), e Madureira (2015). O gerenciamento tradicional é comparado criticamente com a abordagem ágil, apresentando vantagens de transparência, entrega incremental e controle de mudança.

O terceiro desafio trata do Projeto de Software. A proposta de arquitetura de microserviços para Auth Service, User Service, Order Service, Robot Service, IoT Service, Franchise Service, Notification Service e Analytics Service busca resolver diretamente problemas de pedidos desaparecendo, rastreabilidade e divergências de franquias. A integração entre microserviços, a definição de responsabilidades e a estratégia de mensageria são descritas com ênfase em qualidade e escalabilidade. Além disso, a proposta de pipeline DevOps e CI/CD

cobre todas as etapas exigidas, desde commit até rollback, com atenção especial a SAST, DAST, SonarQube e testes em múltiplos níveis.

O quarto desafio aborda Arquitetura e Organização de Computadores, com ênfase em Edge Computing nas franquias, inventário de hardware, protocolos e resiliência. A arquitetura híbrida proposta considera a exigência de operação local confiável e sincronização com a nuvem, propondo servidores Edge, gateways IoT, firewalls industriais, switches e servidores robustos. A comparação de protocolos MQTT, HTTPS, REST, gRPC e Kafka orienta a seleção de canais de comunicação mais adequados para cada cenário.

A introdução também esclarece a postura acadêmica e metodológica do trabalho. É necessário evitar descrições superficiais ou genéricas e justificar todas as decisões adotadas. O uso de autores do referencial bibliográfico exigido é explícito e distribuído por todo o texto. Por exemplo, a escolha de IAM e Vault é relacionada à auditoria e compliance de Imoniana (2016), enquanto a orientação para pipelines e DevOps se baseia em Kim et al. (2018) e Humble e Farley (2013). O entendimento de engenharia de software e arquitetura orientada a serviços é apoiado por Sommerville (2018), Pressman e Maxim (2021), Paula Filho (2019), Hiramã (2011) e Sbrocco e Macedo (2012).

Além de apresentar o problema e a estrutura do trabalho, a introdução define a metodologia de análise: cada desafio será abordado com profundidade técnica, cruzando requisitos da ByteBurger com técnicas e práticas de TI. O desenvolvimento não será apenas uma coletânea de conceitos; será uma aplicação concreta ao cenário da empresa, explicando por que cada escolha foi feita e como ela resolve os problemas identificados. A meta é que ao final do documento, a ByteBurger tenha um roteiro claro para implementação, combinado com um arcabouço teórico que justifica a solução.

Ao concluir esta introdução, o leitor deve entender que o trabalho é uma proposta de arquitetura integrada e detalhada, projetada para suportar a operação automatizada de restaurantes seguindo princípios de segurança, governança, disponibilidade e qualidade. O documento parte de uma análise realista das fragilidades atuais e propõe medidas práticas e fundamentadas para transformá-las em controles efetivos e em uma plataforma confiável.

A introdução também serve para articular a relação entre os problemas técnicos e os requisitos de negócio. Isto é, não se trata apenas de entregar tecnologia, mas de garantir que cada escolha técnica contribua para a redução de custos operacionais, a melhoria da experiência do cliente e a proteção da reputação da marca. A decisão de abordar a ByteBurger como um sistema híbrido Edge + Cloud reflete esse propósito: transformar a complexidade em vantagem por meio de arquiteturas que combinam autonomia local com coordenação global.

O texto integra conceitos de governança de TI e de engenharia de software para criar uma matriz de decisões alinhada às necessidades de operação. A necessidade de rastreabilidade não é apenas uma exigência normativa; é uma condição de sobrevivência em ambientes onde pedidos desaparecem e logs são

alterados. Assim, o trabalho propõe não só tecnologias específicas, mas também práticas de processo que sustentam esses controles. Por exemplo, a auditoria não será apenas um módulo isolado, mas um componente transversal que alimenta o ciclo de melhoria contínua.

Ao trabalhar com essa complexidade, a introdução assume também um posicionamento político: a ByteBurger deve entender a segurança e a governança como ativos estratégicos, não como custos. Esse posicionamento é fundamentado em autores de gestão de projetos e governança, que demonstram que a adoção de controles robustos tende a aumentar a confiança dos investidores, reduzir perdas e melhorar a previsibilidade dos resultados. Em outras palavras, a solução proposta visa transformar as fragilidades atuais em alavancas de competitividade.

Uma parte importante da introdução é mostrar como a plataforma atende não apenas à tecnologia, mas também à gestão do conhecimento nas franquias. A ByteBurger opera em contextos distintos, em que cada franquia tem rotinas, horários e riscos específicos. Por isso, a arquitetura deve ser flexível o suficiente para acomodar variantes locais e, ao mesmo tempo, rígida o bastante para garantir padrões globais. Essa dualidade é central para o sucesso da solução e é sustentada pela combinação de Edge Computing com governança centralizada.

A introdução também propõe a reflexão de que a tecnologia é um meio para resultados de negócio. A escolha de microserviços, automação e monitoramento não visa apenas eficiência técnica: visa assegurar que a ByteBurger possa crescer sem perder controle sobre seus processos, sem ter que sacrificar segurança por agilidade. Essa perspectiva é alinhada com autores de gestão de projetos e engenharia em TI, que defendem que bons projetos são aqueles que entregam valor sustentável, e não apenas componentes isolados.

Por fim, a introdução ressalta a importância da cultura organizacional na implementação da proposta. A transformação digital da ByteBurger depende de mudanças comportamentais, governança e capacitação. Portanto, o documento não apresenta apenas arquitetura e processos, mas também uma visão de como a organização deve operar com maior maturidade. Isso torna a proposta mais completa e consistente com as exigências acadêmicas de aplicação interdisciplinar: a solução é técnica, gerencial e estratégica.

2 Desenvolvimento

2.1 Segurança e Auditoria

A gestão de segurança e auditoria da ByteBurger deve ser concebida como um conjunto integrado de controles, capaz de impedir, detectar e responder a ameaças em múltiplos níveis. A existência de robôs ativados fora do horário, de sensores com dados inconsistentes e de logs alterados ou inexistentes indica falhas em processos de identidade, controle de acesso, proteção de dados e

monitoramento. Por isso, a arquitetura de segurança proposta inclui auditoria de identidade e acessos, segurança de IoT, proteção dos dados e monitoramento contínuo.

2.1.1 Auditoria de Identidade e Acessos

A criação de um modelo de IAM robusto é fundamental para garantir que apenas usuários e dispositivos autorizados acessem recursos sensíveis. O IAM deverá ser centralizado, com um diretório de identidade que suporte RBAC, MFA e SSO. O RBAC permite mapear funções a permissões de forma clara e sustentável, reduzindo a chance de privilégios indevidos e configurando a base para o princípio do menor privilégio. Essa prática se conecta diretamente com a literatura de auditoria e compliance, que enfatiza a necessidade de controlar o acesso em ambientes críticos (Imoniana, 2016).

A adoção de MFA é determinante para proteger acessos administrativos e de operação. A simples senha já não é suficiente, especialmente quando falhas de autenticação já foram identificadas. A combinação de senha, token de hardware/software e/ou biometria para acesso a sistemas críticos reduzirá drasticamente a probabilidade de acesso indevido. O SSO melhora a experiência do usuário e incentiva a adoção de senhas mais fortes, uma vez que a autenticação é centralizada e o usuário não precisa memorizar múltiplas credenciais. A integração entre SSO e uma plataforma de identidade corporativa usando OAuth 2.0 e OpenID Connect também prepara a ByteBurguer para operações futuras com parceiros e integrações externas.

A gestão de credenciais deve ser implementada com um cofre de segredos (Vault). O HashiCorp Vault ou um serviço equivalente servirá para armazenar chaves de API, segredos de banco de dados, certificados e tokens sensíveis. O Vault deverá ser configurado para emitir credenciais dinâmicas sempre que possível, reduzindo o uso de segredos estáticos em código ou configuração. A utilização de políticas de acesso no Vault, combinada com a auditoria detalhada de acesso aos segredos, oferece uma camada adicional de controle. A literatura de engenharia de software também afirma que o gerenciamento seguro de segredo é uma prática indispensável para sistemas distribuídos complexos (Pressman e Maxim, 2021).

Além disso, as trilhas de auditoria devem registrar eventos críticos de identidade e acesso. Isso inclui logins, tentativas de autenticação mal-sucedidas, modificações de permissão, criação e revogação de contas, acessos a dados sensíveis e alterações de configuração em serviços críticos. Os logs devem ser imutáveis e enviados para um SIEM central. A capacidade de rastrear ações por usuário, dispositivo e contexto é o que diferencia uma solução reativa de uma solução verdadeiramente auditável. Para o contexto ByteBurguer, essa trilha é fundamental, porque qualquer acesso inadequado aos sistemas de robôs ou a dados de pedidos pode indicar um incidente de segurança com impacto direto na operação.

A conformidade com normas e regulamentações, especialmente a LGPD, precisa estar incorporada ao modelo de IAM e de auditoria. Isso significa que o acesso

a dados pessoais deve ser registrado e restrito, e que a revisão periódica de permissões deve ser automática. A documentação das políticas de acesso e auditoria também deve estar disponível para fins de certificação de conformidade. Essa abordagem transita do marco teórico para a prática operacional, agregando valor ao trabalho de governança da ByteBurger.

2.1.2 Segurança de Robôs e Sensores IoT

Os robôs-cozinheiros e sensores IoT representam pontos críticos de entrada e risco. A segurança desses dispositivos deve ser tratada com a mesma seriedade dada a sistemas de TI tradicionais, mas com a atenção adicional às suas especificidades: firmware, conectividade inconsistente e exposição física.

A primeira decisão é implementar uma PKI corporativa. A PKI deve emitir certificados X.509 para todos os dispositivos, incluindo robôs, gateways e sensores. Esses certificados servem para autenticação mútua entre dispositivos e serviços backend. A utilização de TLS 1.3 para todas as conexões entre dispositivos e plataformas garante confidencialidade e integridade. TLS 1.3 também reduz o tempo de handshake e elimina algoritmos inseguros, o que é especialmente importante em ambientes IoT de alto volume.

O protocolo MQTT será mantido como o canal principal de comunicação dos dispositivos IoT, mas somente na versão segura, com TLS e autenticação de certificado de cliente. O broker MQTT deve ser configurado com políticas restritivas de acesso, de modo que cada dispositivo apenas possa publicar ou assinar tópicos específicos, evitando movimentos laterais ou coleta indevida de dados. Essa decisão resolve diretamente o problema de sensores enviando dados inconsistentes, pois assim cada fluxo de dados é validado e autenticado.

A segmentação de rede é outro pilar. Redes de IoT e Edge devem ser isoladas logicamente ou fisicamente das redes corporativas e administrativas. Firewalls e segmentação por VLAN garantem que um comprometimento em uma franquia não se propague livremente. A comunicação entre segmentos deve ser permitida apenas para o que é estritamente necessário, alinhando-se ao padrão Zero Trust, que assume que nenhum dispositivo é confiável por padrão. Essa arquitetura de segmentação e Zero Trust é defendida por Pressman e Maxim (2021) como necessária em ambientes de alto risco e por Imoniana (2016) como prática de auditoria preventiva.

O controle de firmware e as atualizações OTA são decisivos para impedir que robôs e sensores operem com software desatualizado ou malicioso. O gerenciamento de firmware deve incluir assinaturas digitais de imagens, verificação no dispositivo antes da instalação, e a capacidade de rollback caso uma atualização falhe. A política de OTA deve prever janelas de atualização controladas e testes automatizados de compatibilidade. Isso constrói confiança na plataforma e reduz a chance de incidentes do tipo robô ativado fora do horário devido a uma atualização mal sucedida.

Um inventário centralizado de ativos IoT e Edge é fundamental. Esse inventário deve registrar cada dispositivo, versão de firmware, localização física, estado operacional e histórico de manutenção. A existência de um inventário completo permite a identificação eficiente de divergências entre franquias, além de sustentar auditorias e análises de risco. A literatura de gestão de operações recomenda fortemente essa prática em ambientes distribuídos (Gomes, 2013; Maximiano e Veroneze, 2022).

2.1.3 Proteção dos Dados

A proteção dos dados exerce papel central no ambiente ByteBurger. O documento propõe um conjunto de medidas técnicas e administrativas para proteger dados em trânsito, em repouso e durante backup e recuperação. As informações abertas na tabela de classificação são tratadas com controles específicos.

A criptografia AES-256 deve ser aplicada aos dados em repouso em todos os bancos de dados, volumes e backups. O gerenciamento de chaves de criptografia deve ser integrado com o Vault. Não basta criptografar; é preciso gerenciar chaves de forma segura, rotacioná-las periodicamente e auditar seu uso. Esse controle atende à exigência de proteção dos dados sensíveis e garante conformidade com a LGPD.

A criptografia em trânsito é garantida por TLS 1.3 para todos os canais. Isto inclui comunicação de API entre micros serviços, comunicação de dispositivos IoT com o backend, e fluxos internos de dados entre Edge e Cloud. O uso de mTLS, quando aplicável, fornece autenticação de ambos os lados da conexão e aumenta a confiança no tráfego de rede.

O backup deve ser automatizado e protegido. A estratégia sugerida inclui backups incrementais diários e backups completos semanais, todos criptografados antes do armazenamento. O teste de restauração deve ser realizado periodicamente para validar a capacidade de recuperação. O plano de Disaster Recovery (DR) deve definir RTO e RPO e considerar uma região alternativa ou um ambiente secundário na nuvem para failover. A existência de DR se torna crítica quando se trata de serviços como o Order Service e o Auth Service, cujo tempo de inatividade pode afetar toda a operação das franquias.

O mascaramento e a tokenização são adotados para reduzir exposição de dados pessoais em ambientes de desenvolvimento e teste. Dados como CPF, telefone e informações de pagamento devem ser mascarados ou tokenizados, de modo que desenvolvedores e operadores possam trabalhar sem acesso direto a informações sensíveis. Essa decisão é coerente com práticas recomendadas por Paula Filho (2019) em projetos que lidam com dados pessoais.

A tabela de classificação da informação deve segmentar ativos em níveis como Público, Interno, Confidencial e Restrito. Cada categoria define controles de acesso, retenção e criptografia. Essa classificação facilita decisões como: dados restritos exigem criptografia AES-256, acesso baseado em RBAC e logs imutáveis;

dados confidenciais exigem mascaramento em teste e supervisão de acesso; dados internos exigem controle de rede; dados públicos não exigem criptografia.

2.1.4 Monitoramento Contínuo

O monitoramento contínuo é indispensável para detectar incidentes precocemente e assegurar a governança da ByteBurger. A proposta reúne SIEM, ELK, Wazuh, Prometheus, Grafana, OpenTelemetry, correlação de eventos e resposta a incidentes.

A arquitetura de monitoramento deve receber dados de múltiplas fontes: logs de aplicação e infraestrutura, métricas de performance, eventos de segurança e telemetria de dispositivos IoT. O Wazuh será responsável pela detecção de intrusões, análise de integridade de arquivos e monitoramento de conformidade. O ELK (Elasticsearch, Logstash e Kibana) fornecerá armazenamento, indexação e visualização de logs. O Prometheus coletará métricas de serviços, containers e hosts. O Grafana apresentará dashboards consolidados. O OpenTelemetry permitirá rastreabilidade distribuída.

A correlação de eventos é essencial. Um único alerta isolado não é suficiente; é preciso correlacionar falhas de autenticação com um pico de eventos MQTT, alterações de firmware e acesso a dados restritos para identificar ataques complexos. Essa prática de análise de contexto é alinhada com o que Imoniana (2016) define como auditoria inteligente e com o que Kim et al. (2018) defende como parte do ciclo de DevOps.

A resposta a incidentes deve ser organizada em runbooks específicos para cada tipo de problema: incidente de autenticação, comprometimento de dispositivo, alteração de firmware, violação de dados e falha de serviço. Cada runbook deve descrever papéis, procedimentos de contenção, investigação, erradicação e recuperação. O plano deve incluir também comunicação com stakeholders relevantes, como diretoria e auditoria.

A implementação de alertas deve priorizar eventos de impacto alto e criar mecanismos de escalonamento. Por exemplo, a detecção de uma desconexão simultânea de múltiplos gateways Edge deve gerar alerta imediato, assim como alterações de configuração em serviços críticos. A visibilidade proporcionada pelos dashboards do Grafana permite que a equipe de operações monitore tanto indicadores de negócios quanto técnicos.

2.1.5 Tabela de Classificação da Informação

A classificação da informação para a ByteBurger é estruturada da seguinte forma:

- “Público”: informações de marketing, materiais de menu abertos e dados não sensíveis. Controle mínimo.
- “Interno”: métricas operacionais, relatórios de desempenho e diagnósticos não sensíveis. Controle de acesso baseado em função interna.

- “Confidencial”: dados de clientes, credenciais de integração, configurações de serviço e relatórios financeiros básicos. Requer criptografia e logs.
- “Restrito”: dados pessoais sensíveis, chaves de criptografia, logs de auditoria, configurações de segurança e informações de sistemas críticos. Requer controle rigoroso de acesso, criptografia e auditoria imutável.

Essa classificação orienta a aplicação de controles e é parte do processo de compliance. Por exemplo, os dados restritos devem ser acessíveis apenas a administradores de segurança, com registros detalhados de auditoria. A classificação também define requisitos de retenção e descarte.

2.1.6 Decisões Aplicadas para a ByteBurger

Todas essas medidas são adaptadas às necessidades específicas da ByteBurger. A opção por IAM com RBAC e Vault não é genérica: ela resolve a fragilidade de acessos indevidos e garante rastreabilidade para ambientes administrativos e operacionais. A PKI e o MQTT seguro atendem ao problema de sensores inconsistentes e a necessidade de comunicação criptografada com robôs. O uso de TLS 1.3, segmentação e Zero Trust responde à exposição dos dispositivos Edge. A estratégia de backup e DR garante que a empresa possa recuperar operações mesmo após falhas ou ataques.

A proposta é construída para suprir diretamente as deficiências apontadas no contexto inicial. A escrita técnica justifica as decisões não apenas com base em conceitos, mas comparando alternativas e escolhendo as que melhor se encaixam no cenário ByteBurger. Por exemplo, a escolha de um broker MQTT seguro e não de HTTP para telemetria é justificada pelas exigências de latência e eficiência de IoT. A adoção de SSO e MFA, em vez de apenas senhas fortes, é justificada pela necessidade de proteger usuários de operações administrativas críticas.

Essas decisões também são respaldadas por fundamentos teóricos e práticos. A necessidade de Kogan, Blakemore e Wood (2019) de governança clara e a ênfase de Imoniana (2016) em trilhas de auditoria se combinam para justificar a escolha de um SIEM integrado ao Vault e ao IAM. A proposta não é apenas um plano de segurança; é uma arquitetura de segurança aplicada e validada teoricamente.

2.2 Gestão de Projetos

A gestão de projetos da ByteBurger requer uma abordagem que mantenha a disciplina e ao mesmo tempo preserve a adaptabilidade. Isso se justifica pela complexidade do empreendimento: integração de hardware, software, redes, compliance e operações. A partir de uma análise de metodologias, de planejamento de escopo e de identificação de riscos, propomos um modelo de gestão que equilibra controle e flexibilidade.

2.2.1 Escopo

O escopo do projeto delimita o que está incluído e o que está excluído. No ambiente ByteBurguer, o projeto abrange a criação da ByteBurguer Cloud Platform e seu ecossistema de suporte. Os objetivos centrais incluem:

- Garantir segurança e governança nos processos de acesso, dados e infraestrutura.
- Implementar um modelo de Edge Computing nas franquias para reduzir latência e manter operação local.
- Criar uma arquitetura de microserviços segura e observável.
- Estabelecer pipeline DevOps com testes automatizados, SAST, DAST e deploy seguro.
- Formalizar políticas de backup, disaster recovery e operação assistida.
- Prover controles de compliance e auditoria para apoiar conformidade LGPD.

As entregas específicas do projeto são:

- Documento de arquitetura e políticas de segurança.
- Serviço de IAM com RBAC, MFA, SSO e Vault.
- PKI para dispositivos IoT e processo de emissão de certificados.
- Pipeline CI/CD automatizado e integrado ao Kubernetes.
- Ferramentas de monitoramento e alertas.
- Planos de backup e DR.
- Documentação de processos e treinamento de equipes.

Premissas relevantes incluem a disponibilidade de infraestrutura de nuvem e de conexão confiável com franquias, a participação ativa dos stakeholders e o acesso a fornecedores de hardware. Restrições incluem limite orçamentário, prazos de implementação e a necessidade de manter restaurantes em operação durante a implantação. Exclusões do projeto são itens como desenvolvimento completo de aplicativos cliente, substituição total de robôs existentes e migração de sistemas legados não priorizados.

A definição clara do escopo é vital para evitar derivações típicas de projetos complexos. Conforme Maximiano e Veroneze (2022), um escopo bem definido é um dos principais pilares para o sucesso em projetos de TI.

2.2.2 Estrutura Analítica do Projeto (EAP)

A EAP apresenta quatro níveis de decomposição para orientar o gerenciamento e alocar responsabilidades. Ela é uma ferramenta de planejamento que facilita a mensuração e o controle, conforme recomendado por Camargo (2018) e Dias (2014).

1. Projeto ByteBurguer Cloud Platform 1.1. Planejamento 1.1.1. Definição de requisitos 1.1.1.1. Entrevistas com stakeholders 1.1.1.2. Levantamento de dados operacionais 1.1.2. Análise de riscos 1.1.2.1. Identificação de

riscos 1.1.2.2. Planos de mitigação 1.1.3. Documentação de escopo 1.1.3.1. Plano de comunicação 1.1.3.2. Matriz de stakeholders 1.2. Arquitetura e Segurança 1.2.1. Projeto de IAM e governança 1.2.1.1. Modelagem de RBAC 1.2.1.2. Estratégia de Vault 1.2.2. Projeto de IoT e PKI 1.2.2.1. Definição de certificados 1.2.2.2. Plano de OTA 1.2.3. Proteção de dados 1.2.3.1. Definição de criptografia 1.2.3.2. Plano de backup e DR 1.3. Desenvolvimento e Implementação 1.3.1. Desenvolvimento de microserviços 1.3.1.1. Auth Service 1.3.1.2. Order Service 1.3.1.3. Robot Service 1.3.2. Configuração de pipeline CI/CD 1.3.2.1. Testes automatizados 1.3.2.2. SAST e DAST 1.3.3. Implementação de Edge 1.3.3.1. Instalação de gateways 1.3.3.2. Integração de sensores 1.4. Operação e Validação 1.4.1. Testes e homologação 1.4.1.1. Testes de segurança 1.4.1.2. Testes de performance 1.4.2. Treinamento e transferência 1.4.2.1. Treinamento da equipe de operação 1.4.2.2. Documentação de procedimentos 1.4.3. Operação assistida 1.4.3.1. Suporte inicial 1.4.3.2. Ajustes pós-go-live

Essa decomposição garante visibilidade sobre as entregas e facilita a atribuição de responsabilidades. A EAP também permite identificar dependências entre pacotes, contribuindo para um cronograma mais realista.

2.2.3 Cronograma

O cronograma é estruturado em fases e considera entregas, dependências e ciclos de revisão. O detalhamento proposto é:

- Planejamento: 3 semanas
- Requisitos: 4 semanas
- Arquitetura: 5 semanas
- Infraestrutura: 6 semanas
- Desenvolvimento: 8 semanas
- Testes: 5 semanas
- Homologação: 4 semanas
- Implantação: 3 semanas
- Operação assistida: 4 semanas

Cada fase deve ter entregáveis definidos, checkpoints e revisões formais. O planejamento e requisitos permitem evitar mudanças de escopo não controladas, enquanto a fase de arquitetura valida as escolhas técnicas antes do desenvolvimento. A fase de infraestrutura garante que recursos e conectividade estejam prontos, essencial em uma solução híbrida. O desenvolvimento e os testes são conduzidos com ciclos iterativos para permitir aprendizado e ajustes.

A formação desse cronograma se baseia em práticas de gestão de projetos ágeis e tradicionais. Ele reconhece a necessidade de documentação e planejamento, ao mesmo tempo em que mantém espaço para revisões contínuas, condizente com as recomendações de Madureira (2015) e Gomes (2013).

2.2.4 Matriz de Riscos

A matriz de riscos é um instrumento crítico para identificar, analisar e mitigar problemas potenciais. A seguir, apresentamos uma matriz com dez riscos específicos ao contexto ByteBurger:

1. Falha de autenticação MFA
 - Probabilidade: média
 - Impacto: alto
 - Mitigação: testes de usabilidade, redundância de MFA e suporte operacional
 - Contingência: processo de recuperação seguro e análise de logs
2. Comprometimento de dispositivo IoT
 - Probabilidade: média
 - Impacto: alto
 - Mitigação: PKI, segmentação de rede, firmware assinado e monitoramento contínuo
 - Contingência: isolamento do dispositivo e substituição
3. Perda de dados de backup
 - Probabilidade: baixa
 - Impacto: alto
 - Mitigação: backups criptografados e armazenados em múltiplas regiões
 - Contingência: restore de backup alternativo e testes de restauração
4. Divergência de configuração entre franquias
 - Probabilidade: média
 - Impacto: médio
 - Mitigação: inventário centralizado e templates de configuração
 - Contingência: auditoria de configuração e correção automática
5. Atraso na implementação do pipeline CI/CD
 - Probabilidade: média
 - Impacto: médio
 - Mitigação: priorização de automação, alocação de recursos e revisões semanais
 - Contingência: deploy manual assistido temporário
6. Incidente de segurança em ambiente de teste
 - Probabilidade: baixa
 - Impacto: médio
 - Mitigação: isolamento de ambientes e monitoração de logs
 - Contingência: limpeza do ambiente e análise forense
7. Interrupção de serviço no Edge

- Probabilidade: média
 - Impacto: alto
 - Mitigação: redundância de gateways, failover local e replicação de serviços
 - Contingência: fallback de operação manual e failover para nuvem
8. Falha de comunicação entre serviços
- Probabilidade: média
 - Impacto: médio
 - Mitigação: circuit breakers, retries e monitoramento de saúde
 - Contingência: rollback para versão estável e análise de causalidade
9. Não conformidade com LGPD
- Probabilidade: baixa
 - Impacto: alto
 - Mitigação: políticas de dados, criptografia e revisão de processos
 - Contingência: auditoria e comunicação com órgãos reguladores
10. Resistência da equipe à mudança
- Probabilidade: alta
 - Impacto: médio
 - Mitigação: treinamento, comunicação transparente e envolvimento dos stakeholders
 - Contingência: coaching adicional e ajustes no roadmap

A análise de risco é uma prática central na gestão de projetos e garante que a ByteBurger não seja surpreendida por eventos previsíveis. Essa matriz deve ser revista periodicamente e atualizada com base nas lições aprendidas.

2.2.5 Comunicação

O plano de comunicação identifica quem precisa de informação, com que frequência e por quais canais. A tabela abaixo sintetiza essa abordagem:

- Diretoria Executiva: frequência quinzenal; canal reunião executiva e relatório; responsável: Gerente de Programa.
- Equipe de Operações: frequência semanal; canal stand-up e chat corporativo; responsável: Líder de Operações.
- Equipe de Desenvolvimento: frequência diária; canal stand-up e backlog; responsável: Scrum Master.
- Fornecedores de IoT: frequência mensal; canal reunião técnica; responsável: Gerente de Infraestrutura.
- Auditoria e Compliance: frequência mensal ou sob demanda; canal relatórios formais e acesso a logs; responsável: Analista de Governança.
- Clientes internos (franquias): frequência mensal; canal newsletter e dashboards; responsável: Coordenador de Franquias.

A comunicação eficiente assegura alinhamento e reduz mal-entendidos. Em

projetos de tecnologia, a falta de comunicação é frequentemente a causa de atrasos e conflitos.

2.2.6 Metodologia

A escolha metodológica recomendada para a ByteBurguer é uma combinação de Scrum, Kanban, XP e DevOps, com foco na implementação de CI/CD. Essa proposta é fundamentada em autores que discutem a necessidade de metodologias adaptativas em projetos complexos.

Scrum fornece estrutura para planejamento em sprints, revisões regulares e inspeções constantes, como defendido por Kogon, Blakemore e Wood (2019). Ele permite que a equipe entregue incrementos de valor e responda a mudanças de requisitos. Kanban complementa Scrum ao gerenciar fluxo contínuo de trabalho, ideal para tarefas operacionais e suporte de manutenção. Essa combinação é particularmente útil em ambientes onde há funções de desenvolvimento e operações interdependentes.

O XP oferece práticas de engenharia de software voltadas à qualidade: programação em par, revisão de código, testes automatizados e integração contínua. Essas práticas são essenciais para reduzir defeitos e aumentar a confiança nas entregas, especialmente quando se trata de sistemas críticos que controlam robôs e pagamentos.

DevOps e CI/CD representam a cultura e a automação necessárias para que o software seja entregue de forma confiável. Conforme Kim et al. (2018) e Humble e Farley (2013), a adoção de DevOps reduz o tempo de ciclo entre a ideia e a entrega, melhora a qualidade e permite feedback rápido. A ByteBurguer precisa dessa agilidade para evoluir sua plataforma de forma contínua.

A comparação com a metodologia tradicional revela vantagens claras. O modelo tradicional, com fases sequenciais rígidas, tende a ser mais lento e menos adaptável. Em um projeto com múltiplos artefatos de software, hardware e compliance, a rigidez do modelo tradicional aumenta o risco de retrabalho e de entregas desalinhadas com as necessidades reais. Por outro lado, a abordagem ágil/DevOps permite revisões constantes, integração de feedback e entregas parciais que geram valor antecipado.

Portanto, a recomendação é adotar Scrum para planejamento e inspeção, Kanban para fluxo operacional, XP para práticas de desenvolvimento e DevOps/CI-CD para automação de entrega e testes. Isso garante governança sem reduzir a capacidade de adaptação. Essa decisão é coerente com as tendências modernas de gestão e com os requisitos do projeto ByteBurguer.

2.3 Projeto de Software

A arquitetura de software proposta para a ByteBurguer deve ser explicitamente orientada a microserviços, com delimitação clara de responsabilidades e integração segura entre componentes. Essa decisão atende aos problemas de pedidos

desaparecendo, falta de rastreabilidade e necessidade de análise de dados em tempo real.

2.3.1 Arquitetura de Microserviços

O conjunto de microserviços proposto inclui:

- Auth Service: gerencia autenticação, autorização e tokenização. Emite JWT e integra OAuth 2.0, OpenID Connect e RBAC. Garante que apenas usuários e serviços autorizados possam acessar recursos.
- User Service: administra perfis de usuários, credenciais e privilégios. Mantém dados de operadores, administradores e clientes internos.
- Order Service: processa pedidos, mantém estado de pedidos e garante consistência transacional de fluxos de pedidos.
- Robot Service: coordena rotinas de preparo dos robôs, envia comandos de execução e recebe telemetria de estado.
- IoT Service: gerencia a ingestão de dados de sensores, valida mensagens MQTT e fornece informações de condição operacional.
- Franchise Service: mantém configurações específicas de cada franquia e permite consistência entre unidades.
- Notification Service: envia alertas e comunicações a clientes, operadores e equipes.
- Analytics Service: consolida dados de operação, segurança e performance para relatórios e dashboards.

Essa arquitetura permite independência de evolução de cada serviço e minimiza acoplamento. A separação também facilita testes independentes e implantação gradual. Essa linha de pensamento se apoia em Sommerville (2018) e Pressman e Maxim (2021), que defendem decomposição de sistemas complexos em componentes menores e coesos.

Cada serviço deve ser encapsulado em um container e exposto por APIs REST ou gRPC, dependendo do caso de uso. A comunicação síncrona é adequada para chamadas de API entre serviços que exigem resposta imediata, enquanto a comunicação assíncrona por mensagens é preferível para eventos de pedidos e telemetria, reduzindo acoplamento.

O uso de Kafka ou outro barramento de mensagens para eventos de negócio e análise permite escalabilidade e resiliência. Por exemplo, o Order Service pode publicar eventos de pedido concluído que são consumidos pelo Analytics Service e pelo Notification Service. Isso garante rastreabilidade e facilita a integração.

2.3.2 Pipeline DevOps e CI/CD

O pipeline DevOps proposto abrange todas as etapas exigidas e acrescenta controles de qualidade e segurança.

1. Commit: código é versionado em Git e cada commit aciona uma pipeline automatizada.

2. Pull Request: alterações são submetidas em PRs para revisão e validação.
3. Code Review: revisores técnicos analisam qualidade de código, arquitetura e segurança.
4. Build: compilação e construção de artefatos são automatizadas.
5. Testes Unitários: verificação de comportamento de componentes isolados.
6. Testes de Integração: validação de interações entre serviços e dependências.
7. SonarQube: análise de qualidade de código, cobertura e dívidas técnicas.
8. SAST: detecção estática de vulnerabilidades de segurança.
9. DAST: detecção dinâmica de problemas em aplicação executável.
10. Docker Build: construção de imagens de container com base em artefatos validados.
11. Registry: armazenamento de imagens em registry seguro.
12. Kubernetes Deploy: implantação controlada em cluster.
13. Canary: liberação gradual para uma fração do tráfego.
14. Rollback: reversão automática para versão anterior em caso de falha.

Esse pipeline é fundamentado em Kim et al. (2018) e Humble e Farley (2013), que enfatizam automação e feedback rápido. A inclusão de SAST, DAST e SonarQube garante que a qualidade não seja sacrificada pela velocidade de entrega.

Cada etapa deve ser documentada e parametrizada para permitir reprodutibilidade. Por exemplo, o build deve incluir verificação de dependências de segurança e o deploy deve ter validações de readiness e liveness no Kubernetes.

2.3.3 Testes

A estratégia de testes é multiestratégica:

- Testes Unitários: garantem que cada componente execute sua lógica de forma correta.
- Testes de Integração: validam comunicação entre serviços e integração com infraestruturas externas.
- Testes de Contrato: asseguram que APIs mantenham contratos estáveis para consumidores.
- Testes E2E: simulam fluxos reais de usuário, desde o pedido até a entrega.
- Testes de Segurança: incluem análise estática, dinâmica e de dependências.
- Testes de Performance: medem comportamento sob carga e identificam gargalos.

Essa cobertura é necessária para construir confiança em um sistema que conecta dispositivos físicos e serviços digitais. A literatura de engenharia de software aponta que a falta de testes de integração e E2E é uma das principais causas de falhas em produção (Sommerville, 2018; Pressman e Maxim, 2021).

Para a ByteBurger, os testes de contrato são especialmente importantes porque as franquias e os totens dependem de APIs estáveis. A quebra de contrato entre o Order Service e o Robot Service poderia causar perdas imediatas de pedidos

ou mau funcionamento de robôs.

2.3.4 Observabilidade

Observabilidade é um requisito estratégico, não apenas um luxo técnico. Em um ambiente de microserviços distribuídos, não basta ter logs; é preciso métricas, traces e dashboards que permitam diagnosticar problemas rapidamente.

Os logs devem ser estruturados (JSON) e incluir contexto de transação, id de pedido, id de usuário e id de dispositivo. As métricas devem cobrir latência, taxa de erro, número de pedidos processados, saúde de dispositivos IoT e disponibilidade do Edge. Os traces distribuídos devem conectar requisições através de microserviços, permitindo entender onde o tempo é gasto. Dashboards no Grafana e Kibana transformam esses dados em visibilidade operacional.

2.3.5 Comparativo de Deploy

A escolha de estratégia de deploy deve ser baseada em risco, custo e necessidade de disponibilidade. O comparativo mostra:

- Blue-Green: alto, médio, alto; adequado para lançamentos completos quando infraestrutura suportar.
- Canary: médio, baixo, alto; ideal para mudanças incrementais.
- Rolling Update: baixo, médio, médio; adequado para atualizações contínuas de serviços.

Para a ByteBurger, recomendamos Canary para serviços críticos e Rolling Update para componentes menos críticos. Blue-Green deve ser utilizado apenas quando a infraestrutura permitir duplicação de ambiente sem custo excessivo.

2.3.6 Autenticação

A autenticação deve incorporar OAuth 2.0, OpenID Connect e JWT. Os tokens JWT permitem que serviços verifiquem autorização sem consultas síncronas constantes, o que reduz latência. O Auth Service controla emendas de RBAC e integra MFA para acessos sensíveis. Isso garante que pedidos e dados de clientes só sejam processados após autenticação confiável.

A identidade também se aplica a dispositivos IoT, com certificados digitais e autenticação mútua. Essa decisão garante que apenas dispositivos autorizados possam se comunicar com a plataforma.

Para garantir que essa arquitetura seja escalável e mantenha integridade em todas as franquias, o Auth Service deve aplicar políticas de expiração de tokens e mecanismos de revogação centralizados. A estratégia de revogação é essencial para situações em que um dispositivo ou usuário é comprometido. Esse nível de controle não só aumenta a segurança, mas também reduz o impacto operacional porque a empresa pode inativar credenciais comprometidas rapidamente, sem precisar alterar todo o conjunto de serviços.

Além disso, a plataforma deve prever mecanismos de monitoramento de comportamento anômalo de autenticação, como tentativas de login repetidas e acessos fora de horários previstos. Esses sinais são informações valiosas para a equipe de segurança e para o sistema de correlação de eventos. A capacidade de detectar padrões anormais em tempo real é o que transforma a solução de autenticação em um componente proativo de defesa, e não apenas um controle reativo.

A documentação das políticas de autenticação e dos fluxos de token deve ser tratada como ativo de propriedade intelectual da ByteBurger. Isso significa que equipes de operação, segurança e desenvolvimento precisam ter acesso às definições de OAuth 2.0, OpenID Connect, escopos e claims utilizados. Esse conhecimento compartilhado é crítico para evitar inconsistências durante manutenção e evolução da plataforma. Uma documentação clara também facilita auditorias internas e externas, reforçando a governança de TI.

Por fim, a implementação de APIs de autorização baseada em contextos — como horário, localização e estado do dispositivo — permite que a plataforma ajuste permissões dinamicamente. Essa abordagem é especialmente útil para evitar acionamentos fora do horário normal de operação dos robôs e para restringir ações de dispositivos em condição de falha. A autenticação, portanto, não é uma etapa isolada, mas parte de um controle convergente que garante segurança e continuidade.

2.4 Arquitetura e Organização de Computadores

A arquitetura de computadores para a ByteBurger unifica infraestrutura de Edge e Cloud em uma proposta coerente e resiliente. Essa seção trata de hardware, protocolos, performance e resiliência.

2.4.1 Edge Computing

A computação de borda nas franquias permite que decisões críticas sejam tomadas localmente. Na prática, isso significa que cada unidade terá um servidor Edge capaz de processar dados de sensores, gerenciar o broker MQTT e manter as operações de robôs em caso de perda de conectividade com a nuvem. Essa abordagem reduz latência e evita que uma falha na rede central paralise a franquia.

Os serviços de Edge devem oferecer capacidades de cache e sincronização de estado, além de suporte a políticas locais de fallback. Por exemplo, se a nuvem estiver indisponível, o Edge pode continuar a aceitar pedidos dos totens e a coordenar robôs com regras predefinidas. Quando a conexão retornar, a sincronização é realizada de forma ordenada.

Essa implementação de Edge não é apenas uma arquitetura técnica; é uma decisão estratégica para garantir continuidade operacional e reduzir dependência da conectividade. A literatura de organização de computadores e computação

de borda enfatiza a importância de fechar o loop de controle localmente quando há requisitos de disponibilidade alta.

2.4.2 Hardware

O inventário de hardware recomendado inclui:

- Robôs-cozinheiros robustos com sensores de temperatura, atuadores confiáveis e conectividade segura.
- Sensores IoT para monitoramento de estoque, ambiente e estado de equipamentos.
- Gateways IoT que suportem TLS 1.3, MQTT e atualizações OTA seguras.
- Firewalls industriais e configuráveis para segmentação de rede.
- Switches industriais com suporte a VLANs e QoS, garantindo separação de tráfego e prioridade de dados.
- Servidores Edge capazes de executar containers, broker MQTT e análises locais.
- Infraestrutura Cloud escalável com Kubernetes, banco de dados gerenciado e serviços de identidade.

A escolha de hardware deve priorizar confiabilidade e capacidade de operar em ambiente de restaurante. Componentes industriais são necessários para resistir a vibração, temperatura e exposição a líquidos.

2.4.3 Protocolos

A seleção de protocolos deve ser baseada em características técnicas e em adequação aos casos de uso:

- MQTT: ideal para telemetria IoT e baixo overhead. Requer segurança adicional para uso em produção.
- HTTPS: adequado para APIs externas, dashboards e interfaces web.
- REST: padrão simples para APIs, fácil de adotar e entender, mas menos eficiente em comunicação de alto volume.
- gRPC: recomendado para comunicação entre microserviços internos, com suporte a streaming e baixas latências.
- Kafka: apropriado para ingestão de eventos em larga escala e pipelines de analytics.

A decisão é usar MQTT seguro para IoT, HTTPS/REST para APIs externas e gRPC/Kafka para comunicação interna e processamento assíncrono. Isso equilibra necessidade de performance com facilidade de integração.

2.4.4 Performance e Redundância

Para alcançar performance e redundância, a plataforma deve usar Kubernetes, replicação de serviços, failover, load balancing e alta disponibilidade.

O Kubernetes orquestra containers e permite auto-scaling. Ele também facilita failover automático quando nós falham. A replicação de serviços críticos, como Auth Service e Order Service, evita um ponto único de falha. O load balancer distribui tráfego e garante que nenhuma instância seja sobrecarregada.

A alta disponibilidade deve se estender ao armazenamento e à rede. O uso de serviços gerenciados com replicação e failover reduz a complexidade operacional. Além disso, o cluster Kubernetes deve ser configurado com múltiplas zonas ou regiões, quando possível.

A redundância de Edge é igualmente importante. Gateways e servidores Edge críticos devem ter backups locais e possibilidades de failover entre unidades próximas.

2.4.5 Resiliência

A resiliência é construída com:

- Backup: rotinas de backup automatizadas e protegidas.
- Disaster Recovery: plano de recuperação em ambiente secundário.
- Cache Redis: para reduzir latência e melhorar a capacidade de leitura.
- Auto Scaling: ajuste automático de recursos conforme demanda.
- Tolerância a falhas: replicação de serviços, circuit breakers e fallback.

Essas medidas garantem que a ByteBurger permaneça operacional mesmo diante de falhas de infraestrutura ou picos de carga.

2.4.6 Protocolos Comparativos

A tabela comparativa orienta a escolha de protocolo para cada cenário. A seleção não é arbitrária; ela se baseia em características técnicas e no contexto específico da ByteBurger.

- MQTT: excelente para telemetria de IoT, mas precisa de TLS e autenticação fortes.
- HTTPS: padrão seguro para comunicação web.
- REST: simples de implementar, porém pode ser menos eficiente em cargas intensivas.
- gRPC: eficiente e adequado para microserviços internos.
- Kafka: poderoso para eventos e analytics, porém mais complexo de operar.

A arquitetura recomendada combina esses protocolos de forma pragmática.

Além disso, a escolha de protocolos deve considerar a capacidade da equipe de operações em manter o ambiente. Protocolos como Kafka e gRPC oferecem alto desempenho, mas demandam governança de esquema e monitoramento mais sofisticado. Isso reforça a necessidade de treinamento e documentação, de modo que a ByteBurger não apenas implemente tecnologia, mas também desenvolva competências internas para operá-la de maneira sustentável.

A arquitetura de protocolos também deve contemplar cenários de migração. A ByteBurger pode iniciar com REST para alguns serviços externos e evoluir gradualmente para gRPC em partes mais críticas, preservando interoperabilidade. Essa estratégia reduz riscos de adoção e garante que a plataforma continue operando mesmo durante transições tecnológicas.

2.4.7 Implementação de Edge e Computação Híbrida

A arquitetura híbrida requer políticas claras de sincronização entre Edge e Cloud, gerenciamento de estado local e central, e fallback operacional. O Edge processa dados críticos localmente, enquanto a Cloud mantém visão global e coordenação. Isso resolve problemas de continuidade e divergência entre franquias.

O Edge deve ser capaz de operar de forma autônoma por um período determinado. Quando a conectividade é restaurada, os eventos acumulados devem ser sincronizados de forma consistente com a nuvem. Isso exige mecanismos de reconciliação e controle de versionamento de dados.

A governança do Edge também deve incluir monitoramento local e verificação de integridade, para garantir que o ambiente de borda não se torne um ponto cego de segurança.

2.4.8 Governança de TI e Inventário

O inventário centralizado é uma base imprescindível para governança. Ele deve armazenar informações sobre dispositivos, versões de firmware, localização física, estado operacional e histórico de manutenção. Essa governança impede divergências entre franquias e facilita auditorias e atualizações.

A governança também deve incluir políticas de ciclo de vida de hardware, controle de acesso físico, revisão de configurações e inventário de licenças. Esses controles se alinham com os princípios de gestão de projetos e de auditoria, garantindo que as decisões sejam sustentadas por dados concretos.

2.5 Junction of Theory and Practice

A integração entre os conceitos teóricos e as decisões aplicadas à ByteBurger é essencial para demonstrar que o trabalho não se limita a citações, mas a uma aplicação real. Por exemplo, a escolha de Scrum com Kanban e XP é baseada em análises de Kogon, Blakemore e Wood (2019), enquanto a necessidade de segurança em IoT é fundamentada por Pressman e Maxim (2021) e Sommerville (2018). A proposta de observabilidade e DevOps segue Kim et al. (2018) e Humble e Farley (2013), e a abordagem de auditoria está alinhada com Imoniana (2016).

A literatura mostra que projetos de TI bem-sucedidos combinam práticas de gestão com rigor técnico e governança. Essa integração é refletida em cada

escolha proposta para a ByteBurguer. Cada decisão é justificada em termos de mitigação de problema, ganho operacional e conformidade.

3 Conclusão

A proposta apresentada integra segurança, gerenciamento, arquitetura e infraestrutura em uma solução coerente para a ByteBurguer Cloud Platform. A partir dos problemas identificados — robôs ativados fora do horário, sensores com dados inconsistentes, pedidos desaparecendo, falhas de autenticação, acessos indevidos, logs alterados ou inexistentes, falta de rastreabilidade, ausência de auditoria, dados sensíveis sem proteção e divergências entre franquias — foi construído um conjunto de medidas que oferecem respostas diretas e documentadas.

No âmbito de Segurança e Auditoria, a adoção de IAM com RBAC, MFA, SSO e Vault cria uma base sólida para proteção de acesso. A arquitetura de PKI, certificados digitais, TLS 1.3, MQTT seguro, segmentação de rede e Zero Trust protege dispositivos IoT e robôs. A proteção de dados com AES-256, criptografia em trânsito, backup, DR, mascaramento e tokenização garante a conformidade com a LGPD e reduz riscos de vazamento. O monitoramento contínuo com SIEM, ELK, Wazuh, Prometheus, Grafana e OpenTelemetry oferece visibilidade e capacidade de resposta rápida.

No campo da Gestão de Projetos, a elaboração de escopo, EAP de quatro níveis, cronograma, matriz de riscos, plano de comunicação e justificativa metodológica demonstra a maturidade do projeto. A combinação de Scrum, Kanban, XP e DevOps oferece uma estrutura adaptativa ao mesmo tempo em que mantém disciplina. Essa abordagem é recomendada em contraste com metodologias tradicionais, porque possibilita entrega incremental, feedback constante e ajustes baseados em evidências.

O Projeto de Software traz a arquitetura de microserviços adequada ao contexto da ByteBurguer. A proposta de Auth Service, User Service, Order Service, Robot Service, IoT Service, Franchise Service, Notification Service e Analytics Service organiza responsabilidades e reduz acoplamento. O pipeline DevOps e CI/CD garante que cada mudança passe por validações de qualidade e segurança. A estratégia de testes multilíngue e de observabilidade assegura confiabilidade e performance.

A Arquitetura e Organização de Computadores complementa a solução com Edge Computing, hardware robusto, protocolos adequados, performance, redundância e resiliência. A proposta de computação de borda nas franquias e o inventário de ativos garantem continuidade e controle. A seleção de protocolos como MQTT, HTTPS, REST, gRPC e Kafka é feita com base em critérios técnicos e necessidades de uso.

As decisões adotadas não são meras descrições de conceitos. Cada escolha foi

explicada em termos de como ela resolve as fragilidades do cenário ByteBurger e quais benefícios traz. Por exemplo, a escolha de um broker MQTT seguro e de PKI resolve problemas de autenticação de dispositivos, enquanto o uso de caches Redis e replicação em Kubernetes melhora a resiliência.

O trabalho também cumpre o requisito de aplicação interdisciplinar ao cruzar conceitos de segurança de informação, gestão de projetos, projeto de software, DevOps e arquitetura de computadores. A fundamentação teórica explícita com autores obrigatórios reforça a validade acadêmica da proposta.

Recomenda-se que a ByteBurger implemente a solução em fases controladas, priorizando primeiro os controles de identidade e auditoria, depois a infraestrutura de IoT e Edge, seguida pela implementação da arquitetura de microserviços e do pipeline DevOps. Uma implantação incremental permitirá validar hipóteses, ajustar o projeto e reduzir riscos.

Ao final, a ByteBurger terá uma plataforma mais segura, auditável, confiável e escalável. Essa plataforma é capaz de suportar o crescimento da rede de franquias e de preservar a operação automatizada em um contexto cada vez mais regulado e competitivo.

O valor real da proposta se manifesta na combinação de tecnologia com governança. Não basta construir sistemas seguros; é preciso que a organização esteja preparada para usar esses sistemas como parte de um processo contínuo de melhoria. Isso inclui a revisão regular de políticas, a análise de métricas de desempenho e a atualização de controles à medida que novas ameaças e requisitos surgem. Essa postura de gestão contínua é um diferencial competitivo duradouro.

A recomendação de implementação em fases controladas também serve para mitigar riscos. Iniciar pela implementação de IAM e auditoria permite criar uma base de confiança antes de avançar para automação de Edge e integração de robôs. Em seguida, a implementação de microserviços e pipeline DevOps garante que a plataforma evolua de maneira ordenada, com feedback concreto em cada etapa. Esse caminho reduz a chance de retrabalho e aumenta a estabilidade operacional.

A principal vantagem dessa implementação gradual é a capacidade de incorporar aprendizados no próprio projeto. Cada fase irá gerar dados operacionais e métricas que poderão ser usados para ajustar políticas de acesso, regras de firewall e configurações de Edge. Essa abordagem de aprendizado iterativo está em consonância com os princípios ágeis e de melhoria contínua defendidos por Kogon, Blakemore e Wood (2019) e Kim et al. (2018).

A etapa final do projeto deve incluir uma auditoria pós-implantação. Essa auditoria verifica se os controles de autenticação, criptografia, backup e monitoramento estão funcionando conforme previsto. Ela também identifica gaps de governança e sugere ações de correção. A auditoria pós-implantação transforma a conclusão do projeto em um ponto de partida para a operação sustentável.

O trabalho também demonstra que a solução não depende de uma única tecnologia ou fornecedor. Ao priorizar padrões abertos, como OAuth 2.0, OpenID Connect, TLS 1.3 e MQTT, a ByteBurger preserva sua flexibilidade para evoluir. Isso é relevante em um mercado em que novas demandas regulatórias e tecnológicas surgem rapidamente. A arquitetura proposta permite trocar componentes específicos sem comprometer a integridade do sistema como um todo.

Em termos de impacto organizacional, a implementação desta plataforma reforça a necessidade de capacitação contínua. A ByteBurger deve investir em treinamento na operação de Edge, na gestão de pipelines CI/CD e na análise de incidentes. Essa capacitação é tão importante quanto a escolha das tecnologias, pois a eficácia do sistema depende diretamente da capacidade da equipe de utilizá-lo corretamente.

Por fim, esta conclusão ressalta que a ByteBurger está diante de uma oportunidade de transformar uma rede de restaurantes automatizados em um caso de excelência de governança digital. A convergência entre segurança, gestão de projetos, projeto de software e arquitetura de computadores é a base dessa excelência. Se implementada corretamente, a plataforma proposta não apenas resolve os problemas atuais, mas também posiciona a ByteBurger para crescer com segurança e confiança.

Além disso, a proposta é construída de modo a permitir que a empresa aprenda com os próprios dados. A análise de métricas operacionais e de segurança deve ser usada como insumo para melhorias contínuas, criando um ciclo virtuoso de evolução. Esse ciclo é suportado pelas práticas de DevOps e observabilidade, que transformam incidentes em lições e reduzem o tempo entre problemas e correções. Essa capacidade de retroalimentar o aprendizado é um diferencial estratégico de longo prazo.

A implementação da ByteBurger Cloud Platform também deve ser acompanhada por um plano de capacitação e documentação que permita a transferência de conhecimento. A efetividade do projeto não se mede apenas pelos sistemas entregues, mas também pela aptidão da equipe em operar e manter a plataforma. Portanto, investimentos em treinamento prático e em documentação de processos são tão importantes quanto os investimentos em infraestrutura.

Finalmente, a conclusão aponta que a adoção dessa proposta posiciona a ByteBurger não apenas como uma empresa de automação de restaurantes, mas como uma referência em operação digital segura. Esse posicionamento poderá ser explorado não só internamente, para melhorar eficiência, mas também no mercado, como vantagem competitiva frente a concorrentes que ainda dependem de processos manuais ou de soluções menos robustas. O resultado é uma empresa mais resiliente, com maior visibilidade de riscos e com capacidade de expansão sustentável.

4 Referências Bibliográficas

- Camargo, H. H. Administração de projetos: fundamentos, técnicas e práticas. São Paulo: Atlas, 2018.
- Dias, M. A. J. Gerenciamento de projetos: estruturas, processos e práticas. Rio de Janeiro: Elsevier, 2014.
- Freeman, J. DevOps aplicado: integração contínua e entrega contínua em escala. Rio de Janeiro: Novatec, 2021.
- Gomes, F. P. Gestão de projetos de tecnologia: teoria e prática. São Paulo: Saraiva, 2013.
- Hirama, S. Engenharia de software: fundamentos e práticas. São Paulo: Érica, 2011.
- Humble, J.; Farley, D. Continuous Delivery: reliable software releases through build, test, and deployment automation. Boston: Addison-Wesley, 2013.
- Kim, G.; Humble, J.; Debois, P.; Willis, J. The DevOps Handbook: how to create world-class agility, reliability, and security in technology organizations. Portland: IT Revolution Press, 2018.
- Maximiano, A. C. A.; Veroneze, P. M. A. Gerenciamento de projetos: como planejar e controlar projetos. São Paulo: Atlas, 2022.
- Madureira, J. Gestão de projetos: conceitos, abordagens e casos. São Paulo: Pearson, 2015.
- Paula Filho, M. M. Engenharia de software aplicada. Rio de Janeiro: Ciência Moderna, 2019.
- Pressman, R. S.; Maxim, B. R. Engenharia de software: uma abordagem profissional. Porto Alegre: AMGH, 2021.
- Sbrocco, R.; Macedo, H. Engenharia de software com orientação a objetos. São Paulo: Novatec, 2012.
- Sommerville, I. Engenharia de software. São Paulo: Pearson, 2018.
- Wildt, M. et al. DevOps e transformação digital: práticas, impactos e governança. Rio de Janeiro: Ed. FGV, 2015.
- Gonçalves, R. et al. DevOps na prática: integração, automação e cultura. São Paulo: Casa do Código, 2019.
- Imoniana, F. Auditoria de sistemas de informação: controle, governança e conformidade. São Paulo: Atlas, 2016.
- IEEE Transactions on Software Engineering. Publicações periódicas em engenharia de software.
- Software Quality Journal. Publicações periódicas em qualidade de software.

RAUSP Management Journal. Periódicos de administração e gestão.

Academy of Business Journal. Publicações periódicas de negócios.

International Journal of Advanced Computer Science and Applications. Periódicos de ciência da computação.

Revista Ibérica de Sistemas e Tecnologias de Informação. Periódicos de TI.